



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

DIPARTIMENTO DI MATEMATICA
LAUREA TRIENNALE IN INFORMATICA

PROGETTO DI BASI DI DATI

Basi di Dati di un Sistema Informativo per la Gestione di un torneo del
GCC Pokémon

Alessandro Adami, Alberto José Toniolo

Indice

1	Abstract	2
2	Analisi dei Requisiti *DA FARE*	2
3	Progettazione Concettuale	2
4	Progettazione Logica	2
4.1	Analisi delle Ridondanze	3
4.2	Eliminazioni delle generalizzazioni	3
4.3	Schema Relazionale	5
5	Implementazione in PostgreSQL e Definizione delle Query	5
5.1	Definizione delle Query	5

1 Abstract

Questo lavoro sviluppa una base di dati progettata per gestire in modo strutturato e coerente le informazioni relative a un torneo, tenendo conto degli utenti, mazzi usati e eventuali carte che li compongono, restrizioni applicate nel torneo, la piattaforma su cui è realizzato e i risultati dei match. L'obiettivo principale è fornire un sistema che supporti le attività di monitoraggio delle statistiche (percentuale di utilizzo, percentuale di vittoria, ecc.) per ogni carta, fornendo informazioni sul bilanciamento del gioco nel suo complesso, oltre a tenere conto della performance di ogni partecipante (gruppo o individuale).

Nel contesto del torneo, la base di dati distingue l'utente tra squadre e i solitari, gestendo separatamente le informazioni relative a un'eventuale organizzazione eSports, oppure il biglietto per il singolo.

2 Analisi dei Requisiti *DA FARE*

Questa sezione riassume i requisiti a cui deve sottostare la base di dati.

Ospedali. Ogni ospedale è identificato da un codice univoco e contiene le seguenti informazioni:

- Codice univoco che distingue in modo univoco ogni struttura sanitaria.
- Numero di dipendenti (se noto).
- La regione geografica in cui si trova.

Gli ospedali si suddividono in: pubblici e privati.

3 Progettazione Concettuale

Figura 1 mostra il diagramma Entità-Relazione (E-R) che riassume i requisiti descritti in sezione 2.

?? a pagina 6 riassume tutte le entità e relazioni individuate dalla analisi dei requisiti nel diagramma E-R, con i relativi attributi rilevanti dall'analisi. Per le entità, viene anche fornito l'identificatore, che contiene anche relazioni nei tre casi di Reparto, Parto e Ricovero.

Il presente schema E-R non permette di rappresentare direttamente i seguenti vincoli:

- Una carta può essere bannata solo una volta
- L'attributo *Costo_Energia* in Abilità non può essere minore di 0
- L'attributo *Durata* in Trainer ha come valore minimo 1 (dura almeno un turno)
- Per ogni partita, il numero minimo di giocatori è 1 mentre il massimo è 2, ovvero:

$$\forall P \in \text{Partita}, \quad \exists U \in \text{Utente} : \quad 1 < |U| \leq 2$$

4 Progettazione Logica

In questa sezione viene illustrato il processo di "traduzione" dello schema concettuale in uno schema logico, con l'obiettivo di rappresentare i dati in modo preciso ed efficiente. Il primo passo consiste nell'analizzare le eventuali ridondanze nel modello, al fine di ottimizzare la struttura complessiva. Successivamente, si procede con l'eliminazione delle tre generalizzazioni. Infine, viene presentato il diagramma ristrutturato, con una descrizione delle modifiche apportate.

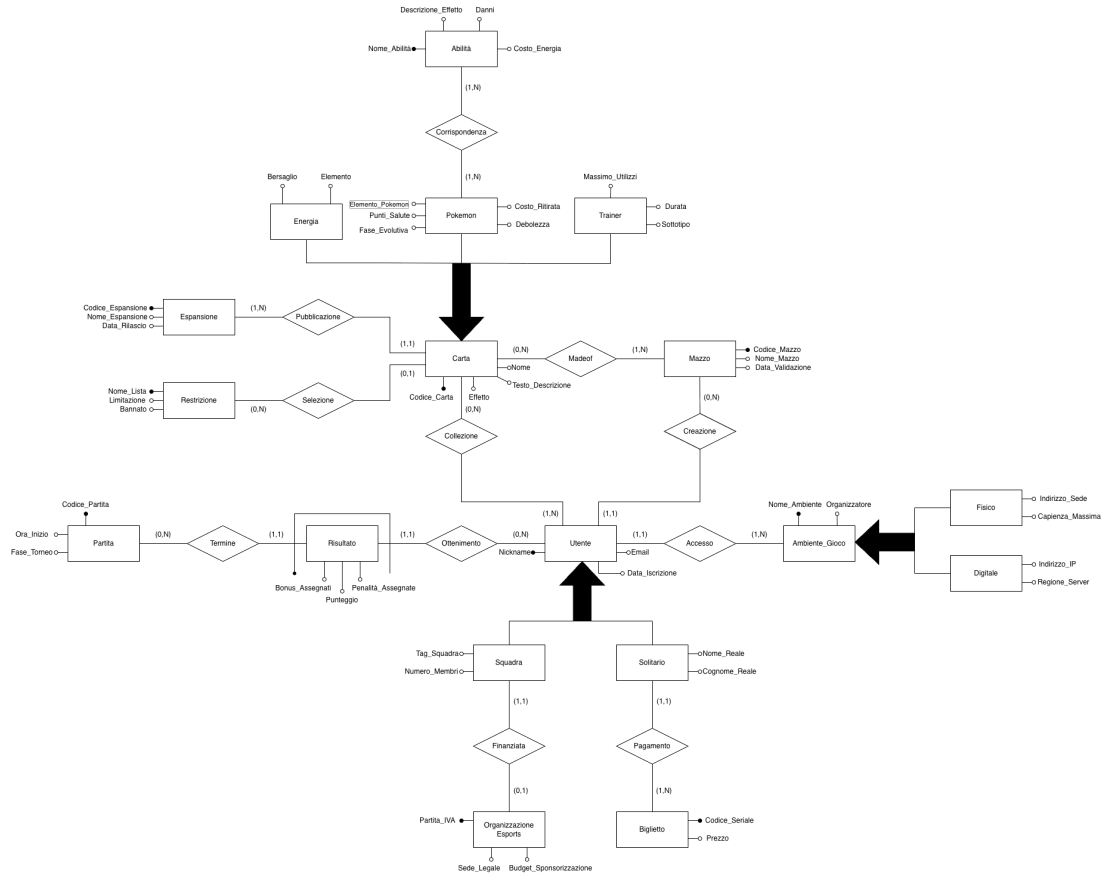


Figura 1: Diagramma E-R della base di dati relativa alla gestione di un torneo.

4.1 Analisi delle Ridondanze

L'attributo N_RICOVERI in REPARTO, che memorizza il numero di pazienti ricoverati nel reparto presenta una ridondanza.

Concetto	Costrutto	Volume
REPARTO	E	50
OSPEDALIZZAZIONE	R	20000
RICOVERO	E	20000

Tabella 1: Assunzione dei volumi nella base di dati

4.2 Eliminazioni delle generalizzazioni

Le generalizzazioni descritte in Sezione 3 vengono eliminate attraverso una ristrutturazione dello schema concettuale, con l'obiettivo di semplificare la successiva implementazione del modello relazionale e ridurre la presenza di valori nulli. Le modifiche vengono applicate come segue:

- **CARTA:** La generalizzazione di Carta viene rimossa includendo le relazioni Is-Poke, Is-Ene e Is-Train, come mostrato in figura 2. Questa scelta riduce la presenza di valori nulli: se si mantenesse un'unica entità CARTA con tutti gli attributi relativi ai tre possibili tipi di una carta, avremmo diversi campi nulli, come i Punti_Salute, Fase_Evolutiva, ecc., mentre ci stiamo riferendo a una carta Trainer (e così anche per gli altri tipi). Separando le informazioni con relazioni dedicate, si garantisce che ogni entità contenga solo gli attributi

rilevanti. In questa soluzione, coerentemente con la metodologia vista a lezione, gli identificatori di Energia, Pokémon e Trainer coincidono con le rispettive relazioni con Carta. Sarebbe stato possibile rimuovere l'entità Carta, incorporandone gli attributi nelle entità figlie. La soluzione sarebbe stata del tutto legittima, ma avrebbe causato la necessità di duplicare la relazione Part-Of in tre relazioni, ovvero verso Energia, Pokémon e Trainer.

- **AMBIENTE GIOCO:** In maniera analoga alla generalizzazione di carta, per Ambiente_Gioco la generalizzazione viene rimossa introducendo le relazioni Is-Fis e Is-Dig. Nello stesso modo mantenendo un'unica entità avremmo valori nulli come Indirizzo_Ip e Regione_Server quando il contesto del torneo è fisico (e viceversa). Sarebbe stato possibile rimuovere l'entità Ambiente_Gioco incorporando gli attributi nelle entità figlie, ma avrebbe anche qui causato la necessità di duplicare la relazione Part-Of in due relazioni.
- **UTENTE:** Anche per Utente la generalizzazione viene rimossa introducendo due relazioni, Is-Squad e Is-Solo. Unendo tutti gli attributi sotto una singola entità Utente porterebbe a campi nulli come Tag_Squadra e Numero_Membri quando ci si riferisce a un utente solitario (e viceversa). Analogamente agli altri due casi, sarebbe stato possibile incorporare gli attributi nelle unità figlie, ma anche qui avremmo duplicato la relazione Part-Of nelle due relazioni, verso Squadra e Solitario.

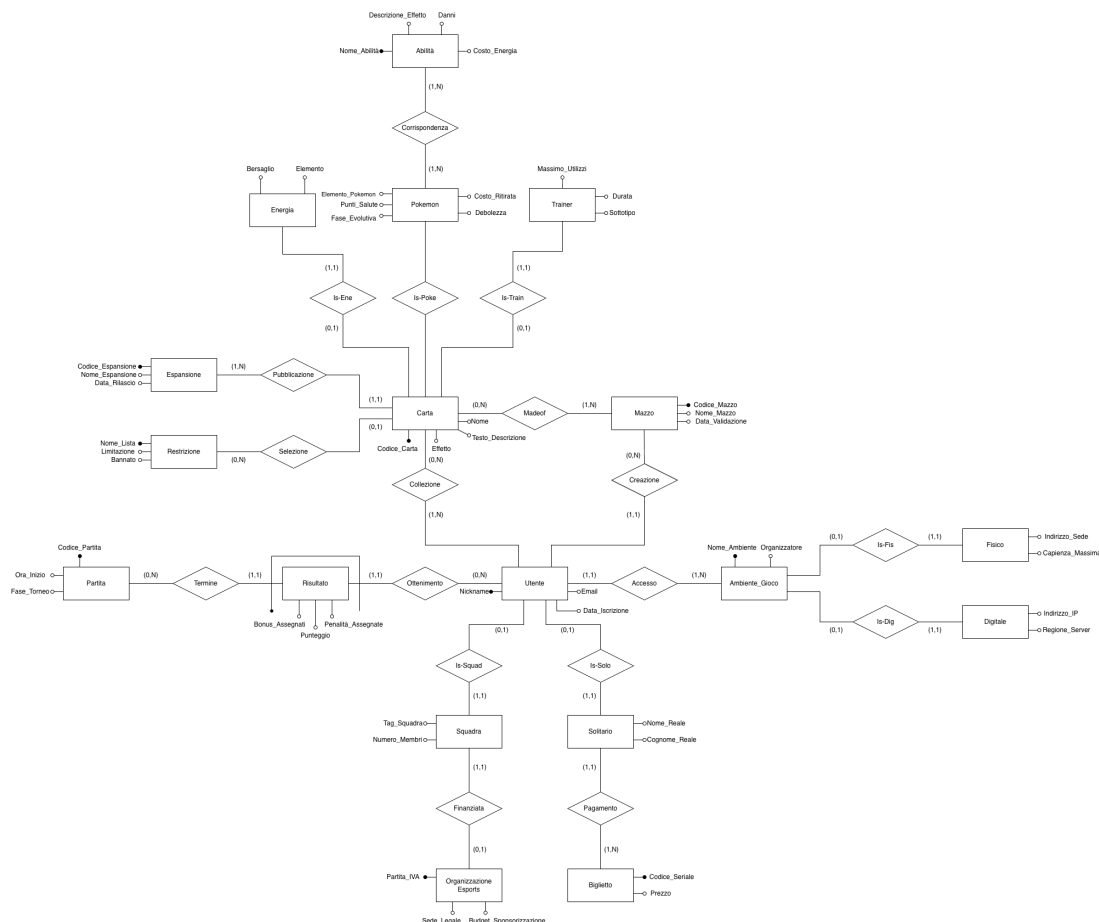


Figura 2: Diagramma E-R ristrutturato.

4.3 Schema Relazionale

Lo schema ristrutturato contiene solamente costrutti mappabili in corrispettivi dello schema relazionale detto anche schema logico. L'asterisco dopo il nome degli attributi indica quelli che ammettono valori nulli.

- **Ospedale**(Codice, N_Dipendenti*, Regione)
- **Privato**(Ospedale, Gruppo)
 - Privato.Ospedale → Ospedale.Codice
- **Dirigente**(Persona_CF, Privato, Ruolo, Data_Inizio)
 - Dirigente.Persona_CF → Persona.CF
 - Dirigente.Privato → Privato.Ospedale

5 Implementazione in PostgreSQL e Definizione delle Query

5.1 Definizione delle Query

Query 1: Contare il numero di reparti per ciascun ospedale stampandoli in ordine decrescente.

```
1 SELECT O.Codice AS CodiceOspedale, COUNT(R.Codice) AS NumeroReparti
2 FROM Ospedale O
3 JOIN Reparto R ON O.Codice = R.Ospedale
4 GROUP BY O.Codice
5 ORDER BY NumeroReparti DESC;
```

Estratto dell'output:

codiceospedale (integer)	numeroreparti (bigint)
1	75
2	4
3	2
4	11